[redacted]

2026

COMP5003: Assignment 1

Contents

Section 2: Discussion & Topic Research 1
Software Design Patterns 1
Computational Complexity 2
Software Development Practices 3
Algorithm Design & Thinking 4
Section 3: Reflection 5
What 5
So What 5
Now What 6
Appendices 8
References 42

Section 2: Discussion & Topic Research

Software Design Patterns

Design patterns are established solutions to common software design problems. They provide templates that make code more reusable, maintainable, and understandable. By using patterns, developers can rely on proven solutions instead of coming up with new solutions, design patterns communicate ideas clearly with others, and create flexible, scalable software. Patterns also help ensure best practices and can simplify the addition of new features. (Startup House, 2026)

Pattern Name	Type	Why Used
Builder	Creational	To construct complex computer objects, making the creation process flexible
Iterator	Behavioural	To provide a way to iterate through a playlist of songs
Adapter	Structural	Using song IDS to work with existing code expecting title-based playback.

In CreationalWindow, the **Builder pattern** is used to assemble a Computer object by setting each component (CPU, GPU, etc.) individually. This pattern is ideal for constructing complex objects with many parts without defining arguments straight away, making the code easier to read and reducing the risk of errors during object creation.

In BehaviouralWindow, the **Iterator pattern** is implemented for traversing a playlist where each song points to the next. This pattern abstracts the process of iterating through a collection (the playlist), so the client code can process each song sequentially without knowing the underlying structure.
In StructuralWindow, the **Adapter pattern** bridges the gap between the old SimpleMusicPlayer interface (which plays songs by title) and the new AdvancedMusicPlayer (which plays by ID). The adapter translates song titles to IDs, allowing legacy code to work with the new system without modification.

In conclusion, the use of these three design patterns not only improves the internal quality of the program but also makes it easier to maintain and extend in the future.

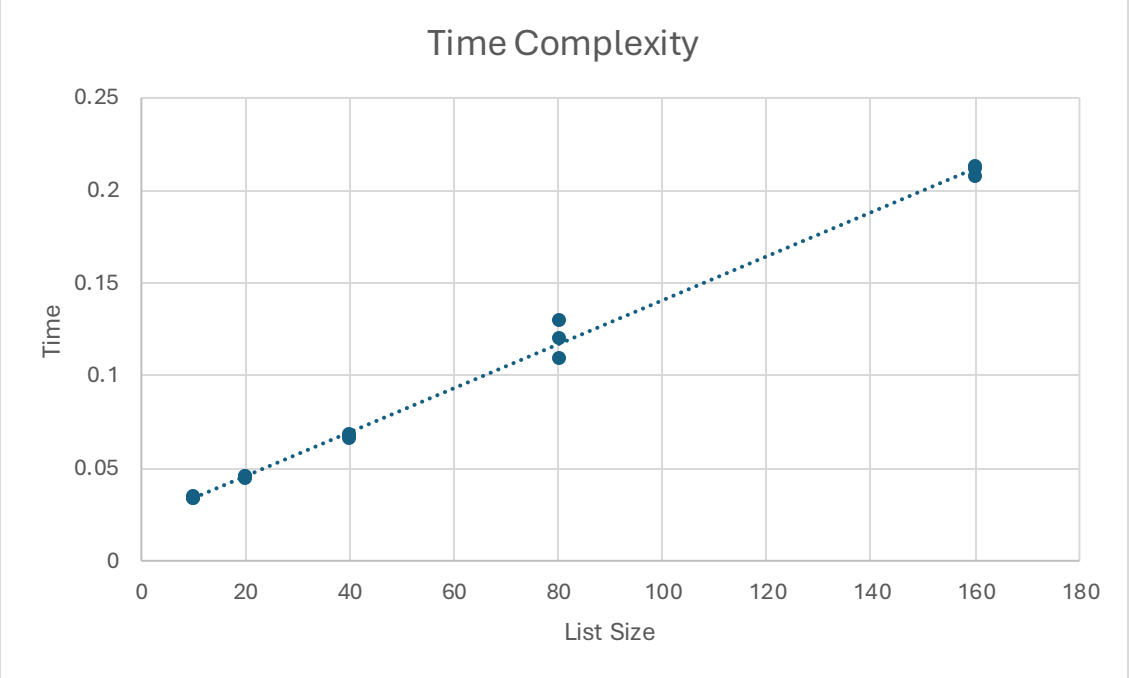
Computational Complexity

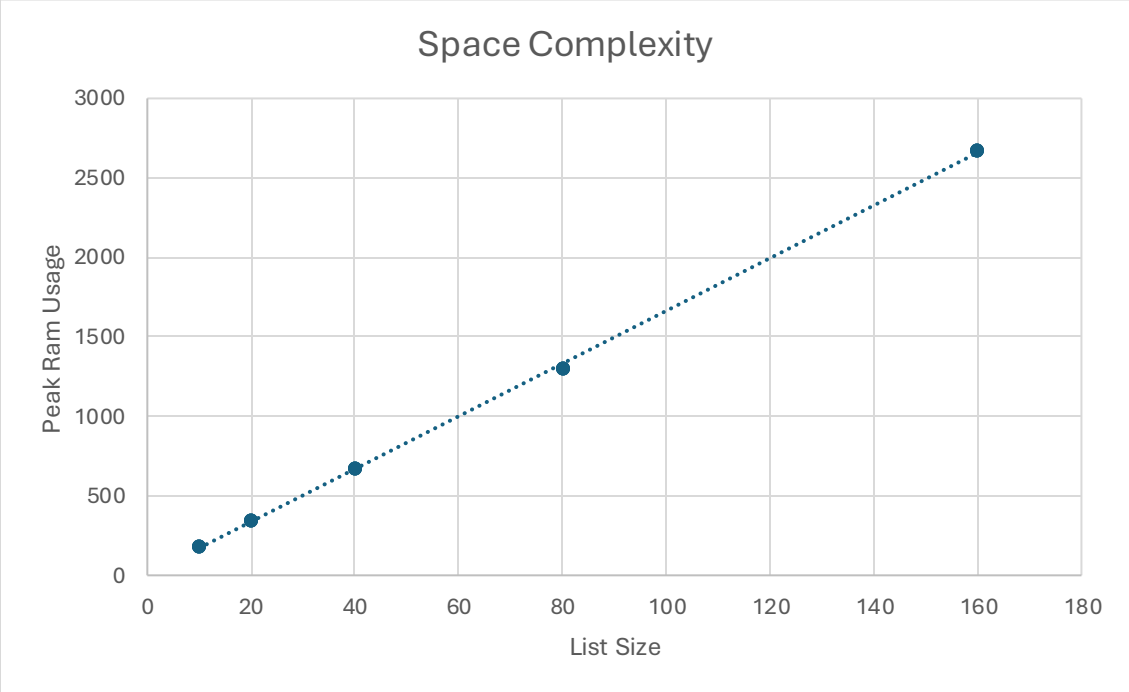
Computational Complexity is a fundamental concept in computer science, which refers to the number of resources, required to solve a given problem using an algorithm. Understanding computational complexity helps computer scientists and programmers evaluate how an algorithm will perform as the amount of input data grows larger. The most widely used method for expressing computational complexity is Big O notation which provides an understanding of how an algorithm's resource requirements grow as the input increases enabling comparisons of the efficiency of different algorithms. (Van, 2025)

For example, the Merge Sort algorithm, implemented in Section 1 of this portfolio, has a time complexity of O(n log n), which is considered quite efficient for sorting large datasets. Merge Sort divides the data into smaller sections, sorts them independently, and then merges the sorted sections together. This divide-and-conquer approach allows Merge Sort to handle even very large lists efficiently, making it a popular choice in many real-world scenarios.

Although the most appropriate algorithm isn't necessarily the one with the best complexity; for example, in cybersecurity, allocating resources for added protection or Machine learning for extra accuracy is common. Likewise for smaller inputs, selection sort is much simpler and easier to work with; despite having much worse complexity, it is much easier to implement for simple small datasets. Highlighting the importance of choosing the correct tool for the job.

Merge Sort		
Input Size	Time	Space
10	0.034	184
10	0.034	184
10	0.035	184
20	0.045	344
20	0.046	344
20	0.046	344
40	0.068	664
40	0.066	664
40	0.067	664
80	0.11	1304
80	0.13	1304
80	0.12	1304
160	0.213	2672
160	0.208	2672
160	0.212	2672





Software Development Practices

Throughout the development of this portfolio, I prioritized best practices to ensure high quality, maintainability, and clarity. All code was managed via GitHub, a version control system, which streamlined rollbacks, commits, file management, and development. By using modular programming, I structured the GUI with a base class that contains common parameters inherited by all algorithm pages, greatly enhancing code readability and reusability.

Additionally, defining all algorithms within classes or functions made the codebase more organized and facilitated easier testing and future updates.

User experience and accessibility were also prioritized during the GUI design. The colour scheme remains consistent throughout, inspired by contemporary applications such as Discord and Instagram. Comprehensive error handling ensures users receive clear feedback, with intuitive widgets that display errors and offer guidance on how to resolve them. These design choices make the portfolio both visually appealing and highly usable for a wide range of users.

Testing played a crucial role in the development process. Where possible, I implemented unit tests to verify critical functions and maintain code reliability. While some algorithms were challenging to test due to their iterative nature and continuously changing values, I focused on validating key outputs and edge cases to ensure overall robustness. I also used debugging tools in my development environment to trace issues and improve performance. Beyond automated tests, I regularly ran the application manually to detect unexpected behaviour and experience the interface as an end user would. This hands-on approach was instrumental in identifying subtle usability issues and making targeted improvements to the overall project.

Algorithm Design & Thinking

This section outlines the design considerations that guided my approach for each algorithm prior to their implementation. I aimed for clarity, modularity, and ease of use, beginning with a main window featuring a dropdown menu for algorithm selection. When the user clicks 'Confirm', a match-case statement opens a dedicated window for the chosen algorithm. This modular approach separates each algorithm into its own class, making the code more efficient and easier to read.

Sorting algorithms (Bubble, Selection, Merge Sort) accept numbers separated by whitespace, which are split and converted to integers. This intuitive input method allows any list length. Try-except blocks provide user-friendly error messages for invalid inputs. Each sorting function accepts an equality parameter ('+' or '-') to toggle ascending/descending sorts, avoiding code duplication.

Fibonacci and factorial algorithms require non-negative integers and provide clear error messages for invalid input. Fibonacci uses memoization. Factorial is implemented recursively. The statistics algorithm calculates median, mode, and interquartile range, reusing selection sort logic to order input. For lists with multiple modes, all highest-frequency numbers are returned.

The RSA function enforces distinct prime inputs, checking for primality and uniqueness to prevent errors. Ciphertext is a space-separated list of integers, and error handling ensures only valid input is processed.

The palindrome algorithm counts all palindromic substrings using two-pointers and memoization for efficient performance. Overall, each algorithm and window was built with a focus on user experience, input validation, and error prevention.

A key consideration for the shuffle algorithm was efficiently representing the full deck without manually defining each of the 52 cards. By defining the Suit and Rank lists separately and generating all possible combinations, the program dynamically creates the complete deck. This approach, combined with randomization, ensures a properly shuffled result.

Section 3: Reflection

What

During the development of this project, I implemented a range of software design patterns—specifically the Iterator, Adapter, and Builder patterns—to structure the code for various algorithmic functionalities within a Python GUI application. These patterns were chosen to promote modularity, maintainability, and clarity in complex sections of the application, including playlist traversal, integration with different music player interfaces, and construction of customizable computer objects.

Alongside the use of design patterns, I adopted several fundamental software development practices. These included version control with Git, thorough code commenting and documentation, functional and performance testing, and iterative development. All algorithms were implemented from scratch, with a focus on readability, adherence to best practices, and user experience within the GUI.

So, What

The deliberate use of software design patterns had a notable impact on the quality of my code and the overall project structure. For example, the Iterator pattern provides a clean, extensible way to traverse a playlist, abstracting away the details of the underlying data structure. The Adapter pattern enabled seamless integration of different interfaces, demonstrating how legacy code and new modules can coexist. The Builder pattern simplified the construction of complex objects, reducing the risk of errors and making the code easier to extend and modify in the future.

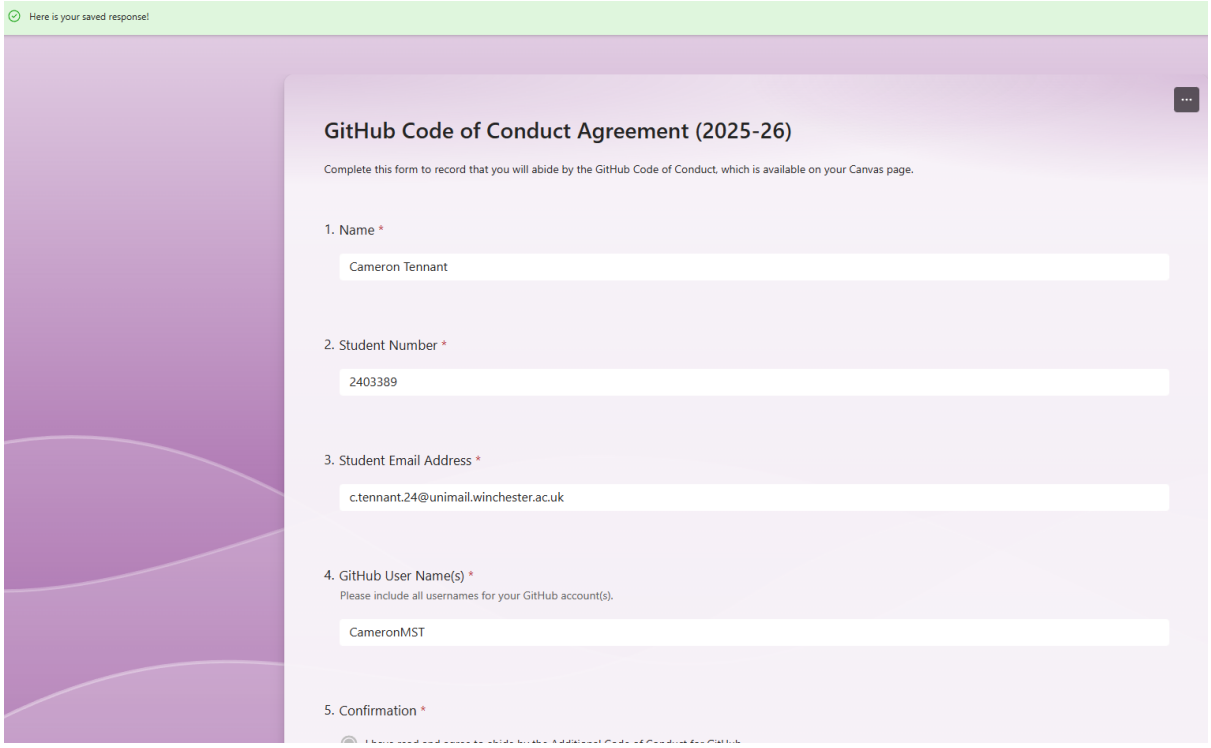
Employing development practices significantly improved the reliability and maintainability of my application. Version control provided a safety net, allowing me to experiment and revert changes if necessary. Testing helped to identify and fix bugs early, ensuring that each algorithm performed as expected and met the assignment requirements. Through iterative development, I refined both the algorithms and the user interface, resulting in a more polished, user-friendly application.

Now What

Reflecting on this process, I recognize that integrating design patterns and best development practices is essential for building scalable, maintainable software, especially as projects grow in complexity. In future assignments, I intend to further expand my repertoire of design patterns and apply them more proactively during the planning phase, rather than awkwardly fitting them in as solutions to emerging problems. I also see value in adopting automated testing frameworks and continuous integration tools to streamline testing and deployment, thereby enhancing reliability and facilitating collaboration in larger teams.

Appendices

https://github.com/CameronMST/Assignment5003-Semester1.git



Here is your saved response!

GitHub Code of Conduct Agreement (2025-26)

Complete this form to record that you will abide by the GitHub Code of Conduct, which is available on your Canvas page.

1. Name *

Cameron Tennant

2. Student Number *

2403389

3. Student Email Address *

c.tennant.24@unimail.winchester.ac.uk

4. GitHub User Name(s) *

Please include all usernames for your GitHub account(s).

CameronMST

5. Confirmation *

```
import time
from tkinter import *
from tkinter import ttk
import ctypes


class BaseWindow: #Setting up a BaseWindow, so other Windows
will inherit the colouring and sizing.
    def WindowParameters(self, window):
        #Window Configuration
        self.title("Algorithm App")
        window.geometry("550x550")
```

```
        window.config(background="#1F1F1F", relief=SUNKEN,
bd=5)

        window.resizable(False, False)

        #Taskbar & Taskbar Image

        window.iconbitmap(r"Winchester.ico")

        myappid = "mycompany.myproduct.subproduct.version"

        ctypes.windll.shell32.SetCurrentProcessExplicitAppUser
ModelID(myappid)


class MainWindow(Tk, BaseWindow):

    def __init__(self): #Constructor

        super().__init__()

        self.WindowParameters(self)


        #Background Image & Text.

        image = PhotoImage(file=r"Winchester.png")  # Opens
Image and Assigns it to variable

        self.photo = image

        banner_Label = Label(self, text="Algorithm App",
font=('Arial', 20, 'bold'), fg='green', bg='#0A0A0A',
image=self.photo, compound='top')

        banner_Label.place(anchor=N, rely=0, relx=0.5)


        # Drop Down Menu

        self.clicked = StringVar()

        self.clicked.set("Select Option")

        drop = OptionMenu(self, self.clicked, "RSA", "Bubble
Sort", "Selection Sort", "Nth Fibonacci Solver", "Merge sort -
Divide and Conquer", "Randomised Algorithm", "Recursion",
"Search Algorithm", "Palindrome" ,"Behavioural Pattern",
"Creational Pattern", "Structural Pattern")  # Source -
https://www.geeksforgeeks.org/python/tkinter-optionmenu-
widget/
```

```
        drop.config(font='Arial', fg='white', bg='purple',
activebackground='purple', activeforeground='white', bd=1,
highlightthickness=0)

        drop.place(anchor=N, rely=0.7, relx=0.5)


        # Confirm Button

        confirm_Button = Button(self, text='Confirm!',
width=12, fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=self.selected_algorithm)  #
You can technically put everything that uses .config on the
same line

        confirm_Button.config(state=ACTIVE)

        confirm_Button.place(anchor=N, rely=0.76, relx=0.5)


    #Switch statements, for opening new Windows

    def selected_algorithm(self):

        selected = self.clicked.get()

        match selected:

            case "RSA":

                RSAWindow(self)

            case "Bubble Sort":

                BubbleWindow(self)

            case "Selection Sort":

                SelectionWindow(self)

            case "Nth Fibonacci Solver":

                FibonacciWindow(self)

            case "Merge sort - Divide and Conquer":

                MergeSortWindow(self)

            case "Randomised Algorithm":

                RandomisedWindow(self)

            case  "Recursion":
```

```
                RecursionWindow(self)
            case "Search Algorithm":
                 SearchWindow(self)
            case "Behavioural Pattern":
                 BehaviouralWindow(self)
            case "Creational Pattern":
                 CreationalWindow(self)
            case "Structural Pattern":
                StructuralWindow(self)
            case "Palindrome":
                PalindromeWindow(self)


    #Algorithm Window Initalisation
class RSAWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
        super().__init__(master)
        self.WindowParameters(self)
        self.EntryP = Entry(self, width = 21)
        self.EntryQ = Entry(self, width = 21)
        self.EntryString = Entry(self, width = 21)
        self.LabelP = Label(self, text="Enter prime number
p:", fg='white', bg='#1F1F1F')
        self.LabelQ = Label(self, text="Enter prime number
q:", fg='white', bg='#1F1F1F')
        self.EnteredString = Label(self, text="Enter a string
to Encrypt / Decrypt:", fg='white', bg='#1F1F1F')
        self.ErrorLabel = Label(self, text="", fg='red',
bg='#1F1F1F')
        self.LabelEncryptDecrypt = Text(self, fg='black',
bg='white', width=40, height=5)
        self.LabelEncryptDecrypt.configure(state=DISABLED)
```

```
        self.LabelP.place(x=50, y=30)
        self.LabelQ.place(x=50, y=80)
        self.EntryP.place(x=50, y=50)
        self.EntryQ.place(x=50, y=100)
        self.EntryString.place(x=50, y=150)
        self.EnteredString.place(x=50, y=130)
        self.LabelEncryptDecrypt.place(x=50, y=230)
        self.ErrorLabel.place(x=50, y=170)

        Encrypt = Button(self, text="Encrypt", fg='white',
bg='purple', activebackground='purple',
activeforeground='white',
command=lambda:self.RSAAlgorithm(True))
        Decrypt = Button(self, text="Decrypt", fg='white',
bg='purple', activebackground='purple',
activeforeground='white',
command=lambda:self.RSAAlgorithm(False))
        Decrypt.place(x=150, y=200)
        Encrypt.place(x=50, y=200)


    def RSAAlgorithm(self, EncryptDecrypt):
        self.ErrorLabel.configure(text="")

        def power(base, exponent, m):
            result = 1
            base = base % m
            while exponent > 0:
                if exponent & 1:
                    result = (result * base) % m
                base = (base * base) % m
```

```
            exponent = exponent // 2
        return result

    def isPrime(n):
        if n <= 1:
            return False
        for i in range(2, n):
            if n % i == 0:
                return False;
        return n;

    def modInverse(e, phi):
        for d in range(2, phi):
            if (d * e) % phi == 1:
                return d
        return None

    def generateKeys(p, q):
        n = p * q
        phi = (p - 1) * (q - 1)
        e = 0
        for e in range(2, phi):
            if gcd(e, phi) == 1:
                break
        d = modInverse(e, phi)
        return (e, n), (d, n)

    def gcd(inta, intb):
        while intb !=0:
```

```
                inta, intb = intb, inta % intb
            return inta

        def encrypt(pk, plaintext):
            key, n = pk
            cipher = [power(ord(char), key, n) for char in
plaintext]
            return " ".join(map(str, cipher))

        def decrypt(pk, ciphertext):
            key, n = pk
            try:
                ciphertext = list(map(int,
ciphertext.split()))
                plain = [chr(power(char, key, n)) for char in
ciphertext]
                return ''.join(plain)
            except ValueError:
                self.ErrorLabel.config(text="Ciphertext is not
valid!")
                return

        try:
            Input1 = int(self.EntryP.get())
            Input2 = int(self.EntryQ.get())
        except ValueError:
            self.ErrorLabel.config(text="Please enter only
integers!")
            return

        if Input1 == Input2:
```

```
            self.ErrorLabel.config(text="p and q cannot be the
same!")
            return

        if not isPrime(Input1) or not isPrime(Input2):
            self.ErrorLabel.config(text="One or both numbers
are not prime!")
            return

        if Input1 < 17 or Input2 < 17:
            self.ErrorLabel.config(text="Please enter primes
greater than 16!")
            return

        public, private = generateKeys(Input1, Input2)

        Message = self.EntryString.get()

        if EncryptDecrypt == True:
            Encrypted_Message = encrypt(public, Message)
            self.LabelEncryptDecrypt.configure(state=NORMAL)
            self.LabelEncryptDecrypt.delete("1.0", "end")
            self.LabelEncryptDecrypt.insert("1.0",
Encrypted_Message)
            self.LabelEncryptDecrypt.configure(state=DISABLED)

        else:
            self.LabelEncryptDecrypt.configure(state=NORMAL)
            Decrypted_Message = decrypt(private, Message)
            self.LabelEncryptDecrypt.delete("1.0", "end")
```

```
            self.LabelEncryptDecrypt.insert("1.0",
Decrypted_Message)
            self.LabelEncryptDecrypt.configure(state=DISABLED)


class BubbleWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
        super().__init__(master)
        self.WindowParameters(self)


        self.Selection = Entry(self, width = 35)


        self.LabelSe = Label(self, text="Enter numbers
seperated by whitespace:", fg='white', bg='#1F1F1F')
        self.outputLabel = Label(self, text="", fg='white',
bg='#1F1F1F')


        self.LabelSe.place(x=50, y=30)
        self.Selection.place(x=50, y=50)
        self.outputLabel.place(x=50, y=120)


        AscSortButton = Button(self, text="Sort Ascending",
fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=lambda:
self.RunBubbleSort('+'))
        DesSortButton = Button(self, text="Sort Descending",
fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=lambda:
self.RunBubbleSort('-'))


        DesSortButton.place(x=50, y=80)
        AscSortButton.place(x=150, y=80)
```

```
    def RunBubbleSort(self, equality):
        list_input = self.Selection.get()


        try:
            my_list = list(map(int, list_input.split()))
        except ValueError:
            self.outputLabel.config(text="Please enter only
integers!")
            return



        def bubblesort(my_list):
            for iter in range(len(my_list)-1):
                    for i in range(len(my_list)-iter-1):


                        if equality == '-':
                            if my_list[i] < my_list[i+1]:
                                my_list[i], my_list[i+1] =
my_list[i+1], my_list[i]
                        #Descending - Placed at top for
default Ascending if any other option is entered
                        else:
                            if my_list[i] > my_list[i+1]:
                                my_list[i], my_list[i+1] =
my_list[i+1], my_list[i]
                        #Ascending
            return my_list


        bubblesort(my_list)
        self.outputLabel.config(text=f"Sorted List:
{my_list}")
```

```
class SelectionWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
        super().__init__(master)
        self.WindowParameters(self)

        self.Selection = Entry(self, width = 35)
        self.LabelSe = Label(self, text="Enter numbers
seperated by whitespace:", fg='white', bg='#1F1F1F')
        self.outputLabel = Label(self, text="", fg='white',
bg='#1F1F1F')

        self.LabelSe.place(x=50, y=30)
        self.outputLabel.place(x=50, y=120)
        self.Selection.place(x=50, y=50)

        AscSortButton = Button(self, text="Sort Ascending",
fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=lambda:
self.Run_Selection_Sort('+'))
        DesSortButton = Button(self, text="Sort Descending",
fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=lambda:
self.Run_Selection_Sort('-'))

        DesSortButton.place(x=50, y=80)
        AscSortButton.place(x=150, y=80)

    def Run_Selection_Sort(self, equality):

        list_input = self.Selection.get()
```

```
        try:
            my_list = list(map(int, list_input.split()))
        except ValueError:
            self.outputLabel.config(text="Please enter only
integers!")
            return


        def selection_sort(my_list):
            for iter in range(0, len(my_list)-1):
                min = iter
                for i in range(iter + 1, len(my_list)):


                    if equality == '-':
                        if my_list[i] > my_list[min]:
                            min = i
                    #Descending - Placed at top for default
Ascending, if anyother option is entered.
                    else:
                        if my_list[i] < my_list[min]:
                            min = i
                    #Descending
                my_list[iter], my_list[min] = my_list[min],
my_list[iter]
            return my_list


        selection_sort(my_list)
        self.outputLabel.config(text=f"Sorted List:
{my_list}") # (W3Schools, n.d)


class FibonacciWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
```

```
        super().__init__(master)
        self.WindowParameters(self)

        self.Fibonacci = Entry(self, width = 21)
        self.LabelFib = Label(self, text="Enter a positive
integer:", fg='white', bg='#1F1F1F')
        self.outputLabel = Label(self, text="", fg='white',
bg='#1F1F1F')

        self.outputLabel.place(x=50, y=120)
        self.Fibonacci.place(x=50, y=50)
        self.LabelFib.place(x=50, y=30)

        FibonacciButton = Button(self, text="Generate the Nth
term", fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=self.run_fib_memo)
        FibonacciButton.place(x=50, y=90)


    def run_fib_memo(self):
        try:
            number = int(self.Fibonacci.get())
            if number < 0:
                self.outputLabel.config(text="Please enter
only positive integers!")
                return
        except ValueError:
            self.outputLabel.config(text="Please enter only
positive integers!")
            return

        def fib_memo(n, mem={}):
```

```
            if n in mem:
                return mem[n]
            if n <=1:
                mem[n] = n
            else:
                mem[n] = fib_memo(n-1, mem) + fib_memo(n-2,
mem)
            return mem[n]

        self.outputLabel.config(text=f"The nth term is
{str(fib_memo(number))}")

class MergeSortWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
        super().__init__(master)
        self.WindowParameters(self)

        self.Selection = Entry(self, width = 35)
        self.LabelSe = Label(self, text="Enter numbers
seperated by whitespace:", fg='white', bg='#1F1F1F')
        self.outputLabel = Label(self, text="", fg='white',
bg='#1F1F1F')

        self.LabelSe.place(x=50, y=30)
        self.Selection.place(x=50, y=50)
        self.outputLabel.place(x=50, y=120)

        AscSortButton = Button(self, text="Sort Ascending",
fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=lambda:
self.Run_Merge_Sort('+'))
```

```
        DesSortButton = Button(self, text="Sort Descending",
fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=lambda:
self.Run_Merge_Sort('-'))


        DesSortButton.place(x=50, y=80)

        AscSortButton.place(x=150, y=80)



    def Run_Merge_Sort(self, equality):

        list_input = self.Selection.get()

        try:

            my_list = list(map(int, list_input.split()))

        except ValueError:

            self.outputLabel.config(text="Please enter only
integers!")

            return


        def Merge_Sort(my_list):

            if len(my_list) > 1:

                mid = len(my_list) // 2



                left_half = my_list[:mid]

                right_half = my_list[mid:]


                Merge_Sort(left_half)

                Merge_Sort(right_half)


                left_index, right_index, main_index = 0, 0, 0
```

```
                while left_index < len(left_half) and
right_index < len(right_half):


                    #Descending Order

                    if equality == '-':

                        if left_half[left_index] >
right_half[right_index]:

                            my_list[main_index] =
left_half[left_index]

                            left_index += 1

                            main_index += 1


                        else:

                            my_list[main_index] =
right_half[right_index]

                            right_index += 1

                            main_index += 1

                    #Ascending Order, Placed Below for
Defaulting.

                    else:

                        if left_half[left_index] <
right_half[right_index]:

                            my_list[main_index] =
left_half[left_index]

                            left_index += 1

                            main_index += 1


                        else:

                            my_list[main_index] =
right_half[right_index]

                            right_index += 1

                            main_index += 1
```

```
                while left_index < len(left_half):
                    my_list[main_index] =
left_half[left_index]
                    left_index += 1
                    main_index += 1

                while right_index < len(right_half):
                    my_list[main_index] =
right_half[right_index]
                    right_index += 1
                    main_index += 1 #(Tutorialspoint, 2019)

            return my_list

        self.outputLabel.config(text=f"Sorted List:
{Merge_Sort(my_list)}")

class RandomisedWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
        super().__init__(master)
        self.WindowParameters(self)

        ShuffleDeck = Button(self, text="Shuffle Deck",
fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=self.Shuffle_Deck)
        ShuffleDeck.place(x=50, y=80)

        self.TextRa = Text(self, fg='black', bg='white',
borderwidth=0, highlightthickness=0)
        self.TextRa.place(x=50, y=120, width= 300, height =
400)
```

```
        self.TextRa.config(state=DISABLED)



    def Shuffle_Deck(self):

        import random #Importing Random, because you cannot
"Randomise" without it, it would be shuffled but not
randomised otherwise

        self.TextRa.config(state=NORMAL)

        Type = ['Heart', 'Diamond', 'Club', 'Spade']

        Ranks = ['2', '3', '4', '5', '6', '7', '8', '9', '10',
'Jack', 'Queen', 'King', 'Ace']


        deck = [(Type, Rank) for Type in Type

                for Rank in Ranks] #(Portfolio Courses, 2023)


        seed = random.randint(1, 1000)

        random.seed(seed)


        for i in range(len(deck)):

            j = random.randint(0, len(deck) - 1)

            deck[i], deck[j] = deck[j], deck[i]


        self.TextRa.delete(1.0, END)

        self.TextRa.insert(END, f"The shuffled deck is Deck
is:\n\n")

        for Type, Rank in deck:

            self.TextRa.insert(END, f"{Rank} of {Type}\n")

        self.TextRa.config(state=DISABLED)


class RecursionWindow(Toplevel, BaseWindow):

    def __init__(self, master=None):
```

```
        super().__init__(master)
        self.WindowParameters(self)

        self.RecursionText = Entry(self, width = 35)
        self.RecursionText.place(x=50, y=50)
        self.RecursionLabel = Label(self, text="Enter a
positive integer:", fg='white', bg='#1F1F1F')
        self.RecursionLabel.place(x=50, y=30)
        self.LabelResult = Label(self, text="", fg='white',
bg='#1F1F1F')
        self.LabelResult.place(x=50, y=115)

        FactorialButton = Button(self, text="Generate
Factorial",fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=self.run_factorial)
        FactorialButton.place(x=50, y=90)

    def run_factorial(self):
        try:
            number = int(self.RecursionText.get())
        except ValueError:
            self.LabelResult.config(text="Please enter only
positive integers!")
            return

        if number < 0:
            self.LabelResult.config(text="Please enter only
positive integers!")
            return

        def factorial(number):
```

```
            return 1 if number <= 1 else number *
factorial(number-1)
        self.LabelResult.config(text=f"The factorial of
{number} is {factorial(number)}")




class SearchWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
        super().__init__(master)
        self.WindowParameters(self)

        self.outputLabel = Label(self, text="", fg='red',
bg='#1F1F1F')
        self.Search = Entry(self, width = 35)
        self.SearchLabel = Label(self, text="Enter numbers
seperated by whitespace:", fg='white', bg='#1F1F1F')

        self.SearchLabel.place(x=50, y=30)
        self.Search.place(x=50, y=50)
        self.outputLabel.place(x=50, y=15)

        SearchButton = Button(self, text="Press for
Statistics", fg='white', bg='purple',
activebackground='purple', activeforeground='white',
command=self.run_search)
        SearchButton.place(x=50, y=80)

    def run_search(self):

        self.outputLabel.config(text="")
```

```
        list_input = self.Search.get()
        try:
            my_list = list(map(int, list_input.split()))
        except ValueError:
            self.outputLabel.config(text="Please enter only
integers!")
            return

        TextBox = Text(self, fg='white', bg='#1F1F1F',
width=40, height=10, borderwidth=0, highlightthickness=0)
        TextBox.place(x=50, y=120)
        TextBox.delete("1.0", "end")
        TextBox.configure(state=DISABLED)

        def selection_sort(my_list):
            for iter in range(0, len(my_list)-1):
                min = iter
                for i in range(iter + 1, len(my_list)):
                    if my_list[i] < my_list[min]:
                        min = i
                my_list[iter], my_list[min] = my_list[min],
my_list[iter]
            return my_list

        sorted_list = selection_sort(my_list)

        def Statistical_Search(sorted_list):

            #Smallest & Largest
```

```
            smallest = sorted_list[0]
            largest = sorted_list[-1]
            #Median
            if len(sorted_list) % 2 == 1:
                median = sorted_list[len(sorted_list)//2]
            else:
                median = (sorted_list[len(sorted_list)//2 - 1]
+ sorted_list[len(sorted_list)//2]) / 2.

            #Inter Quartile
            n = len(sorted_list)
            LowerIQF = sorted_list[n // 4]
            UpperIQF = sorted_list[(3 * n) // 4]

            #Mode
            max_count = 0
            modes = []
            for item in sorted_list:
                count = sorted_list.count(item)
                if count > max_count:
                    max_count = count
                    modes = [item]
                elif count == max_count and item not in modes:
                    modes.append(item) #(GeeksforGeeks, 2018)

            TextBox.configure(state=NORMAL)
            TextBox.insert("1.0", f"Smallest Number:
{smallest}\n")
            TextBox.insert("2.0", f"Largest Number:
{largest}\n")
```

```
            TextBox.insert("3.0", f"Median: {median}\n")

            TextBox.insert("4.0", f"Lower Inter Quartile
Factor: {LowerIQF}\n")

            TextBox.insert("5.0", f"Upper Inter Quartile
Factor: {UpperIQF}\n")

            TextBox.insert("6.0", f"Mode(s): {modes[:]}\n")

            TextBox.configure(state=DISABLED)


        Statistical_Search(sorted_list)





class PalindromeWindow(Toplevel, BaseWindow):

    def __init__(self, master=None):

        super().__init__(master)

        self.WindowParameters(self)


        self.Palindrome = Entry(self, width = 35)

        self.LabelResult = Label(self, text="", fg='white',
bg='#1F1F1F')

        self.PalindromeLabel = Label(self, text="Enter a
string:", fg='white', bg='#1F1F1F')


        self.Palindrome.place(x=50, y=50)

        self.PalindromeLabel.place(x=50, y=30)

        self.LabelResult.place(x=50, y=105)


        PalindromeButton = Button(self, text="Press for
Palindrome Count", fg='white', bg='purple',
activebackground='purple', activeforeground='white',
command=self.run_palindrome)

        PalindromeButton.place(x=50, y=80)
```

```
    def run_palindrome(self):
        InputString = self.Palindrome.get()

        def countPS(InputString):
            string_length = len(InputString)

            memo = [[-1 for i in range(string_length)] for i
in range(string_length)]

            count = 0
            for i in range(string_length):
                for j in range(i + 1, string_length):

                    if isPalindrome(i, j, InputString, memo)
== True:
                        count += 1
            return count
        def isPalindrome(left_index, right_index, InputString,
memo):
            if left_index == right_index:
                return True

            if right_index == left_index + 1 and
InputString[left_index] == InputString[right_index]:
                return True

            if memo[left_index][right_index] != -1:
                return memo[left_index][right_index]
```

```
            if InputString[left_index] ==
InputString[right_index] and isPalindrome(left_index + 1,
right_index - 1, InputString, memo) == True:
                memo[left_index][right_index] = True
            else:
                memo[left_index][right_index] = False

            return memo[left_index][right_index]

        self.LabelResult.config(text=f"Total palindromic
substrings are: {countPS(InputString)}")

class BehaviouralWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
        super().__init__(master)
        self.WindowParameters(self)

        BehaviouralButton = Button(self, text="Run Iterator
Pattern", fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=self.run_iterator_pattern)
        BehaviouralButton.place(x=50, y=80)

    def run_iterator_pattern(self):

        class Song:
            def __init__(self, title):
                self.title = title
                self.next_song = 0

        class Playlist:
            def __init__(self, first_song):
```

```
                self.first_song = first_song
                self.current_song = None

            def __iter__(self):
                self.current_song = self.first_song
                return self

            def __next__(self):
                if self.current_song:
                    title_to_return = self.current_song.title
                    self.current_song =
self.current_song.next_song
                    return title_to_return
                else:
                    raise StopIteration

        song1 = Song("Bohemian Rhapsody")
        song2 = Song("Stairway to Heaven")
        song3 = Song("Hotel California")

        song1.next_song = song2
        song2.next_song = song3

        my_playlist = Playlist(song1)

        self.outputLabel = Label(self, text="", fg='white',
bg='#1F1F1F')
        self.outputLabel.place(x=50, y=120)

        for song_title in my_playlist: #(NeetCode, 2023)
```

```
            self.outputLabel.config(text=f"Now playing:
{song_title}", fg='white', bg='#1F1F1F')
            time.sleep(1)
            self.update()
        time.sleep(2)
        self.outputLabel.config(text="Playlist ended.")









class StructuralWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
        super().__init__(master)
        self.WindowParameters(self)

        DropboxLabel = Label(self, text="Select a Song",
fg='white', bg='#1F1F1F')
        DropboxLabel.place(x=50, y=30)

        self.OutputLabel = Label(self, text="", fg='white',
bg='#1F1F1F')
        self.OutputLabel.place(x=50, y=120)

        selected_song = StringVar()
        dropdown = ttk.Combobox(
            self,
            textvariable=selected_song,
            values=["Bohemian Rhapsody", "Stairway to Heaven",
"Hotel California"],
```

```
            state="readonly"
        )

        dropdown.place(x=50, y=50)
        dropdown.current(0)

        StructuralButton = Button(self, text="Play
Song",fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=lambda:
self.run_adapter_pattern(selected_song))
        StructuralButton.place(x=50, y=80)

    def run_adapter_pattern(self, selected_song):
        #This is harder to demonstrate than my other patterns.

        #Old System
        class SimpleMusicPlayer:
            def __init__(self, output_label):
                self.output_label = output_label

            def playSongByTitle(self, songTitle):
                self.output_label.config(text=f"Playing song:
{songTitle}")

        #New System
        class AdvancedMusicPlayer:
            def __init__(self, output_label):
                self.output_label = output_label

            def playSongById(self, song_id):
```

```
                self.output_label.config(text=f"Playing song
with ID: {song_id}")


        #Adapter
        class TitleToIdAdapter(SimpleMusicPlayer):
            def __init__(self, advancedPlayer, output_label):
                super().__init__(output_label)
                self.advancedPlayer = advancedPlayer
                self.master = master

                self.songLibrary = {
                    "Bohemian Rhapsody": 12455122,
                    "Stairway to Heaven": 99887766,
                    "Hotel California": 55443322
                }

            def playSongByTitle(self, songTitle):
                self.output_label.config(text=f"Adapter:
Translating '{songTitle}' to ID...")
                self.master.update()
                time.sleep(1)

                song_id = self.songLibrary.get(songTitle)

                if song_id:
                    self.advancedPlayer.playSongById(song_id)
                else:
                    self.output_label.config(text=f"Error:
'{songTitle}' not found in library.") #(NeetCode, 2023)
```

```
        master = self
        advanced_player =
AdvancedMusicPlayer(self.OutputLabel)
        adapter = TitleToIdAdapter(advanced_player,
self.OutputLabel)
        adapter.playSongByTitle(selected_song.get())









class CreationalWindow(Toplevel, BaseWindow):
    def __init__(self, master=None):
        super().__init__(master)
        self.WindowParameters(self)


        self.LabelCPU = Label(self, text="Enter CPU Model",
fg='white', bg='#1F1F1F')
        self.LabelGPU = Label(self, text="Enter GPU Model",
fg='white', bg='#1F1F1F')
        self.LabelMotherboard = Label(self, text="Enter
Motherboard Model", fg='white', bg='#1F1F1F')
        self.LabelPSU = Label(self, text="Enter PSU Model",
fg='white', bg='#1F1F1F')
        self.LabelStorage = Label(self, text="Enter Storage",
fg='white', bg='#1F1F1F')
        self.LabelMemory = Label(self, text="Enter Memory",
fg='white', bg='#1F1F1F')


        self.LabelCPU.place(x=50, y=30)
        self.LabelGPU.place(x=50, y=80)
        self.LabelMotherboard.place(x=50, y=130)
```

```
        self.LabelPSU.place(x=50, y=180)
        self.LabelStorage.place(x=50, y=230)
        self.LabelMemory.place(x=50, y=280)



        self.EntryCPU = Entry(self, width = 21)
        self.EntryGPU = Entry(self, width = 21)
        self.EntryMotherboard = Entry(self, width = 21)
        self.EntryPSU = Entry(self, width = 21)
        self.EntryStorage = Entry(self, width = 21)
        self.EntryMemory = Entry(self, width = 21)

        self.EntryCPU.place(x=50, y=50)
        self.EntryGPU.place(x=50, y=100)
        self.EntryMotherboard.place(x=50, y=150)
        self.EntryPSU.place(x=50, y=200)
        self.EntryStorage.place(x=50, y=250)
        self.EntryMemory.place(x=50, y=300)

        BuilderButton = Button(self, text="Build Computer",
fg='white', bg='purple', activebackground='purple',
activeforeground='white', command=self.run_builder_pattern)
        BuilderButton.place(x=50, y=350)

    def run_builder_pattern(self):
        CPUInput = self.EntryCPU.get()
        GPUInput = self.EntryGPU.get()
        MotherboardInput = self.EntryMotherboard.get()
        PSUInput = self.EntryPSU.get()
        StorageInput = self.EntryStorage.get()
```

```
        MemoryInput = self.EntryMemory.get()

        TextBox = Text(self, fg='black', bg='white', width=40,
height=15, borderwidth=0, highlightthickness=0)
        TextBox.place(x=200, y=50)
        TextBox.delete("1.0", "end")

        class Computer:
            def __init__(self):
                self.CPU = None
                self.GPU = None
                self.Motherboard = None
                self.PSU = None
                self.Storage = None
                self.Memory = None

            def setCPU(self, CPUStyle):
                self.CPU = CPUStyle

            def setGPU(self, GPUStyle):
                self.GPU = GPUStyle

            def setMotherboard(self, MotherboardStyle):
                self.Motherboard = MotherboardStyle

            def setPSU(self, PSUStyle):
                self.PSU = PSUStyle

            def setStorage(self, StorageStyle):
                self.Storage = StorageStyle
```

```
    def setMemory(self, MemoryStyle):
        self.Memory = MemoryStyle

class ComputerBuilder:
    def __init__(self):
        self.computer = Computer()

    def setCPU(self, CPUStyle):
        self.computer.setCPU(CPUStyle)
        return self

    def setGPU(self, GPUStyle):
        self.computer.setGPU(GPUStyle)
        return self

    def setMotherboard(self, MotherboardStyle):
        self.computer.setMotherboard(MotherboardStyle)
        return self

    def setPSU(self, PSUStyle):
        self.computer.setPSU(PSUStyle)
        return self

    def setStorage(self, StorageStyle):
        self.computer.setStorage(StorageStyle)
        return self

    def setMemory(self, MemoryStyle):
```

```
                self.computer.setMemory(MemoryStyle)
                return self

            def build(self):
                return self.computer

        computer = ComputerBuilder() \
                    .setCPU(CPUInput) \
                    .setGPU(GPUInput) \
                    .setMotherboard(MotherboardInput) \
                    .setPSU(PSUInput) \
                    .setStorage(StorageInput) \
                    .setMemory(MemoryInput) \
                    .build() #(NeetCode, 2023)
        TextBox.insert("1.0", f"\nComputer
Specifications:\n\n{computer.CPU}\n{computer.GPU}\n{computer.M
otherboard}\n{computer.PSU}\n{computer.Storage}\n{computer.Mem
ory}\n")
        TextBox.configure(state=DISABLED)


app = MainWindow()
app.mainloop()


#References:
```

References

Startup House. (2026). *What is Design Patterns*. [online] Available at: https://startup-house.com/glossary/design-patterns[Accessed 26 Jan. 2026].

Van (2025). *Big O Notation Explained, why is it important?* [online] DEV Community. Available at: https://dev.to/sidge/big-o-notation-explained-why-is-it-important-588p [Accessed 26 Jan. 2026].